

DCN · STUDY GUIDE

The Network Layer: Data Plane

Chapter 4 — Computer Networking: A Top-Down Approach (8th Ed., Kurose & Ross)

Covers: Forwarding vs. routing · router architecture · queuing & scheduling · IPv4/IPv6 · addressing, CIDR, DHCP, NAT · SDN & generalized forwarding · middleboxes

Format: Definitions, key rules, worked examples, watch-outs, and intuition boxes — followed by a consolidated quick-reference sheet

Reorganized for exam-ready review

Table of Contents

4.1	Overview of the Network Layer	03
	Forwarding & Routing — Data and Control Planes	
	Network Service Model	
4.2	What's Inside a Router?	04
	Input Port Processing & Destination-Based Forwarding	
	Switching	
	Output Port Processing & Where Queuing Occurs	
	Packet Scheduling & Net Neutrality	
4.3	The Internet Protocol: IPv4, Addressing, IPv6	07
	IPv4 Datagram Format	
	IPv4 Addressing — Subnets, CIDR, DHCP	
	Network Address Translation (NAT)	
	IPv6	
4.4	Generalized Forwarding & SDN	11
	Match, Action & OpenFlow Examples	
4.5	Middleboxes	12
	The IP Hourglass & the End-to-End Argument	
★	Quick Reference Sheet	13
	All Formulas	
	All Key Definitions	
	Example / Problem Types Covered	

4.1 Overview of the Network Layer

The network layer's job sounds simple — move packets from a sending host to a receiving host — but it splits into two genuinely distinct functions.

Definition

Forwarding — the router-local action of transferring a packet from an input link interface to the correct output link interface. Happens at nanosecond timescales, implemented in **hardware**.

Definition

Routing — the network-wide process that determines the end-to-end paths packets take from source to destination. Happens at second-level timescales, implemented in **software**.

Watch Out

Other authors use "forwarding" and "routing" interchangeably — this textbook does **not**. Keep them precisely separated: forwarding = local per-router action, routing = network-wide path computation.

Intuition

Driving analogy: **forwarding** is like navigating a single highway interchange (car enters, picks an exit). **Routing** is like planning the entire multi-interchange road trip on a map beforehand.

4.1.1 Forwarding and Routing: The Data and Control Planes

Definition

Forwarding table — a router examines header field(s) of an arriving packet and uses these values to index into its forwarding table; the matched entry gives the outgoing interface.

Control Plane — Traditional Approach

A **routing algorithm** runs in every router. Algorithm components across routers exchange **routing protocol** messages to collectively compute each router's forwarding table.

Watch Out

It's hypothetically possible (if unrealistic) for forwarding tables to be configured directly by human operators, with **no routing protocol at all** — this shows forwarding and routing are conceptually separable, even though every real network uses both.

Control Plane — SDN (Software-Defined Networking) Approach

- A **physically separate, remote controller** computes and distributes forwarding tables to every router; the router itself does **forwarding only**.
- The **data plane is identical** in both approaches — only the control plane's location/implementation differs.
- SDN implementations are increasingly **open** (like Linux), letting ISPs, researchers, and students innovate on control logic.

4.1.2 Network Service Model

Key Rules — Possible Service Types

- Guaranteed delivery** — packet eventually arrives
- Guaranteed delivery with bounded delay** — arrives within a delay bound (e.g., 100 ms)
- In-order delivery** — packets arrive in the order sent
- Guaranteed minimal bandwidth** — emulates a fixed-bit-rate link
- Security** — network-layer encryption/decryption

Definition

Best-effort service — the only service model the Internet actually provides: no ordering guarantee, no delivery guarantee, no delay or bandwidth guarantee.

Watch Out

"Best-effort" sounds like a euphemism for no service at all — a network delivering **zero** packets would technically satisfy the definition! Yet combined with good provisioning and adaptive apps (e.g., DASH), it's proven more than good enough for video, VoIP, and conferencing.

Intuition

Alternatives exist — ATM's guaranteed in-order/bounded-delay service, the proposed Intserv [RFC 1633] — but the Internet stuck with simple best-effort and pushed adaptivity into the application layer instead.

Terminology Distinctions

TERM	LAYER	BASES DECISION ON
Packet switch	General	Any device transferring packets input→output based on header fields
Link-layer switch	Layer 2	Link-layer frame fields
Router	Layer 3	Network-layer datagram header fields

4.2 What's Inside a Router?

Four Router Components

1. **Input ports** — terminate the incoming link, do link-layer interop, and perform the **lookup function** (consult forwarding table). Control packets go to the routing processor.
2. **Switching fabric** — connects inputs to outputs; "a network inside the router!"
3. **Output ports** — buffer packets from the fabric and transmit on the outgoing link.
4. **Routing processor** — runs control-plane functions: routing protocols + forwarding-table computation (traditional), or communicates with the remote SDN controller; also handles network management.

Watch Out

"Port" here means a **physical router interface** — not the same as TCP/UDP software port numbers from Chapters 2–3.

Watch Out

At 100 Gbps with a 64-byte datagram, an input port has only **5.12 ns** to process it before the next one may arrive — far too fast for software. Hence input/output ports + switching fabric run in **hardware**; control-plane logic runs in **software** on the routing processor (ms/s timescale).

Intuition

Roundabout analogy: entry road/station = input port (with lookup); the roundabout itself = switching fabric; exit road = output port.

4.2.1 Input Port Processing & Destination-Based Forwarding

Key Rules

1. A **brute-force** table (one entry per possible 32-bit address) is infeasible — over 4 billion possible addresses.
2. Instead, tables use **address-prefix matching**: match against the **longest matching prefix** among entries — the **longest prefix matching rule**.

Worked Example — Prefix Table

PREFIX	INTERFACE
11001000 00010111 00010	0
11001000 00010111 00011000	1
11001000 00010111 00011	2
Otherwise	3

- Destination 11001000 00010111 00010110 10100001 matches the 21-bit prefix of entry 0 → forwarded to **interface 0**.
- Destination 11001000 00010111 00011000 10101010 matches **both** the 24-bit entry (interface 1) and the 21-bit entry (interface 2) → by **longest prefix matching**, goes to **interface 1**.

Watch Out

Lookup at gigabit speeds must happen in **nanoseconds** — ruling out linear search. Routers use specialized hardware like **TCAM** (Ternary Content Addressable Memory), returning an entry in essentially constant time (e.g., Cisco Catalyst 6500/7600 can hold upward of a million TCAM entries).

Definition

Match-plus-action abstraction — (1) match header field(s) against a table, (2) take an action (forward / drop / rewrite). Seen in routers, link-layer switches, firewalls, and NAT. Central to **generalized forwarding** (§4.4).

4.2.2 Switching

Three Switching Techniques

TECHNIQUE	HOW IT WORKS	BOTTLENECK
Via memory	CPU copies packets between input/output buffers via system memory (interrupt-driven). Modern routers (e.g., Cisco Catalyst 8500) do lookup/storing on line cards — only the transfer is shared.	Throughput must be < B/2 (B = packets/sec the bus can read or write — each switched packet needs one read + one write)
Via a bus	Input port prepends a switch-internal label; transmits on a shared bus ; all outputs see it, only the matching one keeps it.	One packet crosses the bus at a time → capped at bus speed (e.g., Cisco 6500's 32-Gbps backplane)
Via interconnection network (crossbar)	2N buses connect N inputs to N outputs; a fabric controller opens/closes crosspoints . Non-blocking — different input/output pairs use different bus pairs (e.g., Cisco 12000). Multistage networks (Cisco CRS) allow more parallelism; capacity also scales by running fabrics in parallel and "spraying" chunks across them.	None between different output ports — but see watch-out below

Watch Out

Even with a crossbar: if **two packets from different input ports are destined to the same output port**, one must still wait — only one packet can cross any single bus at a time.

4.2.3 Output Port Processing

Definition

Output port processing takes packets stored in the output port's memory, performs **scheduling/de-queueing**, then executes link/physical-layer transmission.

4.2.4 Where Does Queuing Occur?

Key Rules

- 1. Queues can form at **both** input and output ports, depending on traffic load and fabric speed vs. line speed.
- 2. If switching fabric rate R_{switch} is **N times faster** than line rate R_{line} (N = number of ports), input-port queuing is negligible.

Input Queueing — Head-of-the-Line (HOL) Blocking

Definition

HOL blocking — a queued packet must wait for transfer through the fabric (even though *its* output port is free) because it's blocked behind another packet at the head of the line.

Watch Out

[Karol 1987]: due to HOL blocking, input-queue length grows **unbounded** (significant loss) once input arrival rate reaches just **58%** of link capacity, under certain modeling assumptions.

Worked Example

Two packets, each at the head of a different input queue, are both destined for the same output port. The switch transfers only one this cycle; the other — and the *lightly shaded packet queued behind it*, destined for a completely different, free output port — must **both** wait. That's HOL blocking in action.

Output Queueing

Watch Out

Even when R_{switch} is $N \times$ faster than R_{line} , output queues can still form: if N input ports simultaneously send to the *same* output port, N packets arrive in the time it takes to transmit just one — the queue can grow until memory is exhausted.

Definition

Drop-tail policy — when the output buffer is full, the newly arriving packet is simply dropped.

Definition

AQM (Active Queue Management) — proactive dropping/marking *before* the buffer is completely full, to signal congestion early. Examples: **RED, PIE, CoDel**.

How Much Buffering Is "Enough"?

Formula — Classic Rule of Thumb [RFC 3439]

$B = RTT \times C$ (B = buffer size, RTT = avg round-trip time, C = link capacity)

Worked Example

A 10-Gbps link with RTT = 250 ms needs $B = 0.25 \text{ s} \times 10 \text{ Gbps} = \mathbf{2.5 \text{ Gbits}}$ of buffering under the classic rule.

Formula — Revised Rule for N Independent TCP Flows [Appenzeller 2004]

$B = (RTT \times C) / \sqrt{2N}$

Watch Out

More buffering is **not** simply "better." Larger buffers cut packet loss but **increase queuing delay** — a 10× buffer increase to reduce loss could increase delay by 10× too, and larger RTTs make TCP slower to react to congestion. "Buffering is a bit like salt — just the right amount makes a meal taste delicious... too much makes it inedible."

Definition

Bufferbloat — long, persistent queuing delay caused by an over-large buffer combined with TCP's self-clocking ACK behavior, even with **no other competing traffic**.

Worked Example — Bufferbloat [Nichols 2012]

Transmission time/packet = 20 ms; RTT = 200 ms. At $t=0$, a burst of 25 packets arrives. One packet is sent every 20 ms; at $t=200$ ms the first ACK returns just as the 21st packet transmits, triggering the sender to release a new packet that queues behind the rest. This ACK-clocked replenishment keeps the queue **persistently at 5 packets** — the link stays fully utilized, yet delay is constantly inflated, frustrating users (e.g., gamers) with zero competing traffic. **DOCSIS 3.1** added an AQM mechanism [RFC 8033/8034] to fight this.

4.2.5 Packet Scheduling

DISCIPLINE	BEHAVIOR
FIFO / FCFS	Packets transmitted in exact arrival order. If buffer is full, a packet-discarding policy decides whether to drop the new arrival or evict a queued one.
Priority Queuing	Classified into priority classes on arrival (e.g., network-management or VoIP > e-mail). Scheduler always serves the highest-priority non-empty queue. Non-preemptive — once a transmission starts, it's never interrupted.
Round Robin	Strict rotation among classes (1, 2, 1, 2...) regardless of priority. Work-conserving — never idles the link if any class has packets; skips an empty class immediately.
WFQ (Weighted Fair Queuing) [Demers 1990; Parekh 1993]	Generalized round robin: class i gets weight w_i , guaranteed a fraction $w_i/\Sigma w_j$ of service (over active classes, and even worst-case over <i>all</i> classes).

Formula — WFQ Guaranteed Throughput

$\text{throughput}_i \geq R \times (w_i / \sum w_j)$ (R = link rate)

Worked Example — Two-Class Round Robin

Packets 1, 2, 4 = class 1; packets 3, 5 = class 2. Packet 1 transmits immediately; meanwhile 2 (class 1) and 3 (class 2) queue. After packet 1, scheduler looks for class 2 → sends 3. Then looks for class 1 → sends 2. Packet 4 is now the only one queued, so it goes next, right after packet 2.

Watch Out

WFQ's idealized description ignores **packetization** — a packet isn't actually interruptible mid-stream to serve another class exactly proportionally. [Demers 1990; Parekh 1993] address this practical nuance.

Principles in Practice — Net Neutrality

2015 FCC "Open Internet" Order — Three Bright-Line Rules

1. **No Blocking** — can't block lawful content/apps/services/non-harmful devices (subject to "reasonable network management")
2. **No Throttling** — can't impair/degrade lawful traffic based on content, app, or service
3. **No Paid Prioritization** — can't favor traffic in exchange for payment

Watch Out

Documented violations predate the 2015 Order: a 2005 NC ISP blocked Vonage VoIP to protect its own phone service; in 2007 Comcast injected **TCP RST packets** to interfere with BitTorrent [FCC 2008]. The **2017 FCC Restoring Internet Freedom Order** rolled back the 2015 prohibitions in favor of a transparency-based approach — the debate remains unsettled.

4.3 The Internet Protocol (IP): IPv4, Addressing, IPv6, and More

4.3.1 IPv4 Datagram Format

Version (4)		Header length (4)		Type of service (8)		Datagram length (16)
16-bit Identifier		Flags (3)		13-bit Fragmentation offset		
Time-to-live (8)		Upper-layer protocol (8)		Header checksum (16)		
32-bit Source IP address						
32-bit Destination IP address						
Options (if any)						
Data						

FIELD	PURPOSE
Version (4b)	IP version (4 for IPv4); tells the router how to read the rest
Header length (4b)	Header length in 32-bit words — needed since options make the header variable-length. No-options header = 20 bytes
Type of service	Distinguishes real-time vs. non-real-time; 2 bits reused for ECN
Datagram length (16b)	Header + data, in bytes; theoretical max 65,535 bytes ; in practice rarely >1,500 (Ethernet payload limit)
Identifier / flags / frag. offset	Support IP fragmentation . IPv6 does not support fragmentation at all
TTL	Decrement by 1 at every router; dropped at 0 — prevents infinite circulation from routing loops
Protocol (8b)	Which transport protocol gets the payload (6 = TCP, 17 = UDP) — the "glue" binding network and transport layers, like a port number binds transport and application
Header checksum (16b)	1's-complement sum, covers only the header ; must be recomputed at every router (TTL changes in transit). Failed-checksum datagrams typically discarded
Source / destination IP	32 bits each
Options	Rarely used; complicate header-length determination and processing time; removed entirely in IPv6
Data	Typically a TCP/UDP segment, but can carry other data (e.g., ICMP)

Watch Out

TCP/IP checks errors at **both** transport and network layers because: (1) only the IP *header* is checksummed at the IP layer, while TCP/UDP checksums the *entire* segment; and (2) TCP doesn't always run over IP (e.g., it can run over ATM), and IP doesn't always carry TCP/UDP.

Worked Example

A datagram carrying a TCP segment has **40 bytes** total header overhead (20 IP + 20 TCP), assuming no options.

4.3.2 IPv4 Addressing

Definition

Interface — the boundary between a host/router and one of its physical links. An **IP address is associated with an interface**, not the host or router as a whole.

Each IP address is **32 bits** (~4 billion possible), written in dotted-decimal notation (e.g., `193.32.216.9`).

Worked Example — Binary/Decimal

`193.32.216.9` in binary is `11000001 00100000 11011000 00001001`

Definition

Subnet (IP network) — detach each interface from its host/router, creating isolated network "islands"; interfaces terminate the endpoints.

Worked Example — Subnet Identification

Three host interfaces + one router interface sharing the leftmost 24 bits (`223.1.1.xxx`), interconnected without an intervening router, form subnet `223.1.1.0/24` . The full network has three such subnets: `223.1.1.0/24` , `223.1.2.0/24` , `223.1.3.0/24` .

Worked Example — Subnets from a Router Mesh

Three routers (R1, R2, R3) pairwise interconnected, each also serving 2 hosts. "Detach each interface" produces **six** subnets total: 3 host-facing (`223.1.1.0/24` , `223.1.2.0/24` , `223.1.3.0/24`) **plus 3 router-to-router** (`223.1.9.0/24` R1-R2, `223.1.8.0/24` R2-R3, `223.1.7.0/24` R3-R1).

Definition

CIDR (Classless Interdomain Routing) [RFC 4632] — address form `a.b.c.d/x` : leftmost x bits = **network prefix**, remaining 32–x bits distinguish devices within the org.

Watch Out

Routers **outside** an org only need to consider the leading x prefix bits — this enables **address aggregation** (route summarization), dramatically shrinking external forwarding-table sizes.

Classful Addressing (Pre-CIDR)

Watch Out

Network portion was constrained to exactly **8, 16, or 24 bits** ("classful" A/B/C). A **Class C (/24)** supports only $2^8 - 2 = 254$ hosts (2

reserved) — too small for many orgs. A **Class B (/16)** supports 65,534 hosts — far more than an org with ~2,000 hosts needs, **wasting >63,000 addresses** per such allocation — a key driver of address exhaustion that CIDR fixed.

Worked Example — Address Aggregation

Fly-By-Night-ISP advertises a single prefix `200.23.16.0/20` externally, even though 8 separate organizations sit inside that block, each with its own /23. If Fly-By-Night later acquires ISPs-R-Us (`199.31.0.0/16`), and Organization 1 (originally `200.23.18.0/23`) connects via ISPs-R-Us instead, Org 1 **doesn't need to renumber**: ISPs-R-Us simply advertises the more-specific `200.23.18.0/23` alongside its own `/16` . Outside routers apply **longest prefix matching** and correctly route via ISPs-R-Us, even though Fly-By-Night's broader /20 would otherwise also match.

Definition

IP broadcast address — `255.255.255.255` : delivered to all hosts on the same subnet; routers usually do **not** forward it into neighboring subnets.

Worked Example — Hierarchical Allocation

An ISP holding `200.23.16.0/20` divides it into **eight equal /23 blocks** — Org 0 = `200.23.16.0/23` , Org 1 = `200.23.18.0/23` , ..., Org 7 = `200.23.30.0/23` .

Definition

ICANN — global authority managing the IP address space, allocating blocks to **regional Internet registries** (ARIN, RIPE, APNIC, LACNIC), and managing DNS root servers and domain-name assignment/disputes.

Obtaining a Host Address: DHCP

Definition

DHCP (Dynamic Host Configuration Protocol) [RFC 2131] — lets a host automatically obtain an IP address ("plug-and-play" / zeroconf protocol).

Key Rules — DHCP also provides

- 1. The host's **subnet mask**
- 2. The **default gateway** (first-hop router) address
- 3. The **local DNS server** address

DHCP's Four-Step Process

- 1. **DHCP discover** — client broadcasts (UDP, dest port 67, dest IP `255.255.255.255` , src IP `0.0.0.0`) since it knows neither network nor server address
- 2. **DHCP offer(s)** — server(s) broadcast back transaction ID, proposed IP, network mask, and **lease time**
- 3. **DHCP request** — client picks one offer, echoes back its parameters
- 4. **DHCP ACK** — server confirms; client may use the address for the lease duration (renewable)

Watch Out

DHCP's biggest mobility shortcoming: a **new IP address is obtained on every subnet change**, so an ongoing TCP connection **cannot survive** the move — handled separately by mobile cellular mechanisms (Chapter 7), not DHCP.

Intuition

DHCP relay agents (typically routers) forward DHCP messages to a server on a different subnet when no local DHCP server exists.

4.3.3 Network Address Translation (NAT)

Definition

Private address realm — a network whose addresses only matter within that network. [RFC 1918] reserves three blocks (e.g., `10.0.0.0/8`) for this — reused identically across hundreds of thousands of independent networks.

Definition

NAT-enabled router — presents to the outside world as a **single device with one public IP**, hiding the internal private topology.

NAT Operation

- 1. Internal host (e.g., `10.0.0.1`) sends a datagram with arbitrary source port (e.g., 3345) to an external server
- 2. NAT router rewrites the **source IP** to its own WAN address (e.g., `138.76.29.7`) and assigns a **new source port** (e.g., 5001) not already in its **NAT translation table**; adds an entry
- 3. The reply arrives at `138.76.29.7:5001` ; NAT looks up the table to recover the original internal address/port (`10.0.0.1:3345`), rewrites destination fields, forwards into the LAN

Watch Out

Capacity: a 16-bit port field means NAT can in principle support over **60,000 simultaneous connections** per single WAN-side IP.

Watch Out — Architectural Objections to NAT

- 1. Port numbers were designed to address *processes*, not *hosts* — complicates running **servers** or **P2P** peers behind NAT (unsolicited inbound connections have nowhere predetermined to land); addressed by NAT-traversal techniques [RFC 5389]
- 2. "Architectural purists" object that **routers should only process up to layer 3** — NAT violates this by rewriting layer-4 port numbers too, breaking strict end-to-end host-to-host communication

Focus on Security — Firewalls and IDS

Definition

Firewall — inspects datagram/segment headers, denies entry to suspicious traffic (e.g., blocking ICMP echo to thwart port-scanning, filtering by IP/port, or tracking TCP connection state).

Definition

IDS (Intrusion Detection System) — performs **deep packet inspection** (headers + payload), matching a continuously updated **signature database** of known attacks; raises an alert on a match.

Definition
IPS (Intrusion Prevention System) — like an IDS, but **actively blocks** matching packets rather than only alerting.

Watch Out
Firewalls and signature-based IDSs can't fully shield a network — attackers keep devising attacks with no existing signature; they protect only against *known* patterns.

4.3.4 IPv6

Motivation: by the early 1990s, the 32-bit IPv4 space was clearly running low. IANA allocated its **last remaining pool** of unassigned IPv4 addresses to a regional registry in **February 2011**.

Most Important IPv6 Changes

- Expanded addressing** — 32 bits → **128 bits**. Adds **anycast** (delivers to *any one* host in a group — e.g., nearest mirror) alongside unicast/multicast
- Streamlined, fixed 40-byte header** — several IPv4 fields dropped/made optional for faster processing; flexible new options encoding
- Flow labeling** — sender labels packets belonging to a particular flow needing special handling (e.g., non-default QoS); "flow" deliberately left open-ended by RFC 2460

```
Version (4) | Traffic class (8) | Flow label (20)
Payload length (16) | Next header (8) | Hop limit (8)
Source address (128 bits)
Destination address (128 bits)
Data
```

FIELD	PURPOSE
Version	Carries value 6
Traffic class	Analogous to IPv4 TOS — priority among datagrams
Flow label	Identifies a flow of datagrams
Payload length	Bytes after the fixed 40-byte header
Next header	Protocol the payload goes to (same values as IPv4's protocol field)
Hop limit	Decrement per router (analogous to TTL); dropped at 0
Source / dest. address	128 bits each [RFC 4291]

Watch Out
Putting a "4" in the IPv6 Version field does **not** create a valid IPv4 datagram — IPv4 and IPv6 are not interoperable by simple field substitution.

Fields Removed in IPv6 (and why)

- Fragmentation/reassembly fields** — IPv6 disallows in-network fragmentation entirely. A too-large datagram gets **dropped** with a "Packet Too Big" ICMP reply, instead of being fragmented — speeds up router processing
- Header checksum** — transport (TCP/UDP) and link-layer (Ethernet) already checksum; recomputing per-hop (since hop limit changes) was costly for little benefit
- Options field** — moved to a **next-header** type instead, preserving a fixed 40-byte base header

Transitioning from IPv4 to IPv6 — Tunneling

Watch Out
A "**flag day**" (universal simultaneous upgrade) is considered effectively impossible at Internet scale — even 40 years ago, with a far smaller Internet, the comparable NCP→TCP flag day was deemed infeasible [RFC 801].

Definition
Tunneling [RFC 4213] — encapsulating an entire IPv6 datagram as the **payload of an IPv4 datagram**, routed across IPv4-only routers (unaware they're carrying IPv6), then **de-encapsulated** at the receiving IPv6 node — identified via **IPv4 protocol number 41**.

Worked Example
IPv6 nodes B and E want to interoperate but are connected via IPv4-only routers C and D. B encapsulates the IPv6 datagram inside an IPv4 datagram addressed to E. C and D route it as ordinary IPv4 traffic. At E, protocol number 41 reveals an IPv6 payload — E extracts and forwards it as if it arrived directly from an IPv6 neighbor.

Intuition
"Introducing new protocols into the network layer is like replacing the foundation of a house — difficult without tearing the whole house down," unlike application-layer protocols, which are "like adding a new layer of paint — relatively easy." This explains IPv6's slow, multi-decade adoption versus rapid app-layer innovation.

4.4 Generalized Forwarding and SDN

Definition

Generalized forwarding — extends match-plus-action beyond destination-IP-only matching, to matching over **multiple header fields across multiple protocol layers**, with actions like forward, load balance, rewrite headers (NAT-style), drop (firewall-style), or redirect for deep inspection.

Definition

Flow table — OpenFlow's term for the generalized match-plus-action table; computed, installed, and updated by a **remote controller**.

Watch Out

Because forwarding may use **network- and/or link-layer** addresses, devices implementing generalized forwarding are more accurately called **"packet switches"** rather than strictly routers (layer 3) or switches (layer 2).

Each Flow Table Entry Contains

1. **Header field values to match** — hardware-matched, typically TCAM (over a million entries possible [Bosshart 2013]); non-matching packets may be dropped or sent to the controller
2. **Counters** — packets matched, time since last update
3. **Actions** — forward to a port, drop, duplicate to multiple ports, rewrite header fields

4.4.1 Match

Link layer:	Ingress Port Src MAC Dst MAC Eth Type VLAN ID VLAN Pri
Network layer:	IP Src IP Dst IP Proto IP TOS
Transport layer:	TCP/UDP Src Port TCP/UDP Dst Port

Key Rules

1. Matching spans **link, network, and transport** layers simultaneously — deliberately crossing the traditional layering principle (§1.5)
2. **Wildcards** allowed (e.g., `128.119.*.*` matches any address sharing that 16-bit prefix)
3. Each entry has a **priority**; if multiple entries match, the **highest-priority** action wins

Watch Out

Not all IP header fields are matchable in OpenFlow (e.g., neither TTL nor datagram length) — a deliberate functionality-vs-complexity tradeoff, per Lampson's maxim [Lampson 1983]: "Do one thing at a time, and do it well... Don't generalize; generalizations are generally wrong."

Watch Out

OpenFlow 1.0's 12 matchable values has since grown to **41 values** in later specs [Bosshart 2014] — the abstraction keeps expanding.

4.4.2 Action

Possible Actions

1. **Forwarding** — to a specific port, broadcast to all (except arrival port), multicast to a set, or send to the **remote controller** for further processing
2. **Dropping** — an entry with no specified action means drop
3. **Modify-field** — rewrite up to **10** header fields (all of Figure 4.29's fields except IP Protocol) before forwarding

4.4.3 OpenFlow Examples of Match-plus-Action

Worked Example 1 — Simple Forwarding

Goal: packets from h5/h6 to h3/h4 go s3 → s1 → s2 (bypassing the direct s3-s2 link).

s1 Flow Table:	
Match: Ingress Port=1; IP Src=10.3.*.*; IP Dst=10.2.*.*	→ Forward(4)
s3 Flow Table:	
Match: IP Src=10.3.*.*; IP Dst=10.2.*.*	→ Forward(3)
s2 Flow Table:	
Match: Ingress port=2; IP Dst=10.2.0.3	→ Forward(3)
Match: Ingress port=2; IP Dst=10.2.0.4	→ Forward(4)

Worked Example 2 — Load Balancing

Goal: traffic from h3 to `10.1.*.*` uses the direct s2-s1 link; traffic from h4 to the same destination prefix is routed via s2-s3-s1 instead — **not achievable** with plain destination-based IP forwarding, since both flows share the same destination prefix.

s2 Flow Table:	
Match: Ingress port=3; IP Dst=10.1.*.*	→ Forward(2)
Match: Ingress port=4; IP Dst=10.1.*.*	→ Forward(1)

Worked Example 3 — Firewalling

Goal: s2 accepts (on any interface) only traffic originating from hosts attached to s3.

```
s2 Flow Table:
```

```
Match: IP Src=10.3.*.*; IP Dst=10.2.0.3 → Forward(3)
```

```
Match: IP Src=10.3.*.*; IP Dst=10.2.0.4 → Forward(4)
```

If these are the *only* entries, traffic from any other source is implicitly dropped (no matching entry).

Intuition

Flow tables can also implement **virtual networks** — two or more logically separate networks using the same physical switches and links.

Definition

P4 (Programming Protocol-independent Packet Processors) — a richer line-rate language for datagram processing, going beyond match-plus-action with variables, arithmetic/Boolean operations, functions, and conditionals [P4 2020; Bosshart 2014].

4.5 Middleboxes

Definition

Middlebox [RFC 3234] — any intermediary box performing functions apart from a normal, standard IP router's data-path functions between source and destination hosts.

Three Broad Types of Middlebox Services

1. **NAT translation** — rewriting IP addresses and port numbers (§4.3.3)
2. **Security services** — firewalls (block/redirect for DPI), IDS (pattern detection), application-level e-mail filters (block junk/phishing/threats)
3. **Performance enhancement** — compression, content caching, load balancing across a server pool

Definition

NFV (Network Function Virtualization) [Mijumbi 2016] — implementing middlebox services on **commodity hardware** with specialized software, rather than dedicated hardware boxes — mirroring SDN's approach a decade earlier. An alternative: **outsourcing to the cloud** [Sherry 2012].

Watch Out

Middleboxes **violate the clean layer separation** between network layer (routers forwarding by IP header only) and transport/application layers: NAT rewrites network *and* transport fields; firewalls inspect application, transport, *and* network layers; e-mail gateways filter on application content plus IP blacklists. Some call this "a bit of an architectural abomination" [Garfinkel 2003]; others argue middleboxes "exist for important and permanent reasons" and will only proliferate [Walfish 2004].

Principles in Practice — Architectural Principles of the Internet

RFC 1958: *"the goal is connectivity, the tool is the Internet Protocol, and the intelligence is end to end rather than hidden in the network."*

The IP Hourglass

Definition

The **"narrow waist"** of the Internet protocol stack — many protocols exist above (HTTP, SMTP, QUIC, DASH, RTP...) and below (Ethernet, PPP, WiFi, Bluetooth...) the network layer, but only **one universal network-layer protocol — IP** — that every connected device must implement.

Intuition

[Clark 1997]: the narrow waist's purpose is to "hide the detailed differences among various underlying technologies and present a uniform service interface to the applications above" — letting IP run over radically different links (copper, fiber, radio) while presenting one consistent service upward.

Watch Out

As the Internet enters "middle age" (40–50 years), the textbook notes its narrow waist "may indeed be widening a bit... via the rise of middleboxes" — in tension with the historical narrow-waist principle.

The End-to-End Argument

Definition

The end-to-end argument [Saltzer 1984] — a function can be **completely and correctly** implemented only with the knowledge and help of the application at the endpoints; providing it as a feature of the communication system itself is therefore not fully possible (though a lower-layer version may still help as a performance enhancement).

Worked Example — Reliable Data Transfer

Even if link-layer protocols implement hop-by-hop error control, this alone is **incomplete** for true end-to-end reliability — packets can still be lost through mechanisms unrelated to link-level bit errors (e.g., a router crashing while holding a queued packet, or a link failure detaching part of the network). Reliable data transfer must ultimately be implemented end-to-end (i.e., via TCP), even when redundant link-layer error control also exists.

★ Quick Reference Sheet

All Formulas

FORMULA	MEANING
Longest prefix matching rule	When a destination matches multiple table prefixes, the longest (most specific) match wins
$\text{Throughput} < B/2$	Memory-switching limit; B = max packets/sec the shared memory bus can read or write
$B = RTT \cdot C$	Classic router buffer-sizing rule of thumb [RFC 3439]
$B = (RTT \cdot C) / \sqrt{2N}$	Revised buffer-sizing rule for N independent TCP flows [Appenzeller 2004]
$\text{throughput}_i \geq R \cdot (w_i / \sum w_j)$	WFQ's guaranteed minimum throughput for class <i>i</i>
HOL blocking threshold $\approx 58\%$	Of input capacity before unbounded queue growth [Karol 1987]
$2^{16} - 1 = 65,535 \text{ bytes}$	IPv4 max datagram length (16-bit length field)
$20 + 20 = 40 \text{ bytes}$	IPv4 header + TCP header (no options), standard combined overhead
40 bytes	IPv6 fixed header — no options in base header
$2^8 - 2 = 254$	Class C (/24) host capacity (2 addresses reserved)
$2^{16} - 2 = 65,534$	Class B (/16) host capacity
~60,000+ connections	NAT translation-table capacity per public IP (16-bit port field)

All Key Definitions

TERM	ONE-LINE DEFINITION
Forwarding	Local, hardware-speed move of a packet from input to correct output interface
Routing	Network-wide, software-speed computation of end-to-end paths, populating forwarding tables
Forwarding table	Per-router map from header-field values/prefixes to outgoing interfaces
SDN	Control-plane architecture where a remote controller (not each router) computes/distributes forwarding tables
Best-effort service	The Internet's sole network-layer model: no delivery, ordering, delay, or bandwidth guarantees
Input port lookup	Forwarding-table consultation at the input port to choose the output port
Switching fabric	Internal mechanism (memory, bus, or crossbar) connecting input to output ports
HOL blocking	An input-queued packet waits unnecessarily because a packet ahead of it (destined elsewhere) is served first
Drop-tail	Dropping an arriving packet when the output buffer is full
AQM	Proactively dropping/marketing packets before the buffer fully fills (e.g., RED, PIE, CoDel)
Bufferbloat	Persistent excessive queuing delay from an oversized buffer + TCP's self-clocking
FIFO/FCFS	Transmits queued packets in arrival order
Priority queuing	Always serves the highest-priority non-empty class first (non-preemptive)
Round robin	Serves classes in strict rotation, skipping empty ones (work-conserving)
WFQ	Generalized round robin guaranteeing each class a weighted share of bandwidth
Net neutrality	2015 FCC principle prohibiting ISP blocking, throttling, and paid prioritization
Interface	Boundary between a host/router and one physical link; IP addresses attach here, not to the host
Subnet (IP network)	Set of interfaces reachable without passing through a router; shares a common prefix
CIDR	Variable-length network-prefix addressing (<code>a.b.c.d/x</code>) replacing fixed class boundaries
Classful addressing	Pre-CIDR scheme restricting prefixes to exactly 8, 16, or 24 bits (Class A/B/C)
Address aggregation	Advertising one external prefix to represent many internal subnets, shrinking external tables
ICANN	Global authority for IP address-space allocation and DNS root/domain administration
DHCP	Protocol for automatic ("plug-and-play") IP address assignment to a new host
NAT	Lets many privately-addressed devices share one public IP via header rewriting + a translation table
Firewall	Inspects/filters packets or connections by header criteria
IDS	Deep packet inspection against a signature database; alerts on matches

IPS	Like an IDS, but actively blocks matching packets
IPv6	Successor protocol: 128-bit addresses, fixed 40-byte header, flow labeling, no in-network fragmentation
Anycast address	IPv6 address type delivering to any one member of a designated group
Tunneling	Encapsulating an IPv6 datagram inside an IPv4 datagram to cross IPv4-only routers
Generalized forwarding	Match-plus-action over any combination of link/network/transport fields, not just destination IP
Flow table	OpenFlow's match-plus-action table, computed/distributed by a remote controller
OpenFlow	The widely adopted SDN standard defining the match-plus-action flow-table abstraction
P4	Richer line-rate packet-processing language beyond OpenFlow's match-plus-action
Middlebox	Any in-network device (NAT, firewall, IDS, load balancer, cache) beyond standard router forwarding
NFV	Implementing middlebox functions on commodity hardware/software instead of dedicated boxes
IP hourglass	The observation that the Internet has many app- and link-layer protocols, but only one network-layer protocol (IP)
End-to-end argument	Certain functions (e.g., reliability) can only be fully/correctly implemented at the communicating end systems

Example / Problem Types Covered

- Finding the outgoing interface for a destination address using a prefix table, resolving ties via **longest prefix matching**
- Computing input-port processing-time budgets (ns available per datagram at a given link speed/size)
- Comparing switching-fabric architectures (memory, bus, crossbar) and their throughput bottlenecks/blocking conditions
- Diagnosing **HOL blocking** scenarios given a specific arrival pattern across input ports
- Computing buffer size via the classic $RTT \times C$ rule vs. the revised $RTT \cdot C / \sqrt{2N}$ rule
- Tracing a **bufferbloat** scenario from transmission time, RTT, and burst size, to find the resulting persistent queue length
- Tracing FIFO, priority, round-robin, and WFQ scheduling order given a packet-arrival sequence with class labels
- Identifying all subnets in a host/router network diagram (the "detach each interface" exercise)
- Converting IP addresses between dotted-decimal and binary
- Allocating CIDR blocks hierarchically (ISP → client orgs) and reasoning about address aggregation / longest-prefix-match routing (the Fly-By-Night-ISP / ISPs-R-Us scenario)
- Tracing the four-step DHCP discover/offer/request/ACK exchange given a topology with a server and relay agent
- Tracing a NAT router's translation-table behavior for an outbound request and its inbound reply
- Comparing IPv4 vs. IPv6 header fields and explaining why each was dropped/added (fragmentation, checksum, options)
- Tracing an IPv4-tunneled IPv6 datagram's path, identifying what each router/endpoint sees
- Writing/interpreting OpenFlow-style flow-table entries for forwarding, load balancing, and firewalling across multiple switches
- Reasoning about net-neutrality rule violations given a described ISP traffic-management practice